Information Systems Department

Faculty of Engineering – Siliwangi University

Tasikmalaya, 2026

PERTEMUAN 6

Metode Pengembangan Perangkat Lunak: Terstruktur vs. Berorientasi Objek

Konsep Paradigma, Pemodelan Sistem (DFD, ERD, UML), Strategi Pemilihan SDLC, dan Contoh Metodologi

Mata Kuliah: Rekayasa Kebutuhan Perangkat Lunak | Dosen: Erna Haerani, S.T., M.Kom.
Jurusan Sistem Informasi, Fakultas Teknik — Universitas Siliwangi (2026)

Capaian Pembelajaran & Agenda Perkuliahan

Capaian Pembelajaran (Learning Outcomes)

- Memahami filosofi mendasar Pendekatan Terstruktur vs. Berorientasi Objek.
- Menguasai teknik pemodelan terstruktur (DFD & ERD) dan pemodelan UML.
- Mampu menganalisis kriteria objektif untuk menentukan pemilihan metode SDLC.
- Mengevaluasi contoh model SDLC (Waterfall, RAD, Spiral, RUP, Agile) untuk studi kasus.

Agenda Empat Blok Utama

- Blok I (Slide 3-14): Pendekatan Terstruktur, DFD, ERD, Skema Database, & Kode C++/PHP.
- Blok II (Slide 15-28): Pendekatan Berorientasi Objek, Pilar OO, & 9 Diagram Standar UML.
- Blok III (Slide 29-30): Matriks Komparasi Diagnostik & Kriteria Penentuan Pemilihan SDLC.
- Blok IV (Slide 31-33): Pembahasan Model SDLC Klasik, Iteratif, Agile, & Tugas Studi Kasus.

Konsep SDLC & Peran Metodologi Pengembangan

Mengapa Perlu Metodologi?

• Pengurangan Risiko

Mencegah biaya pembengkakan akibat kesalahan analisis awal.

• Komunikasi Efektif

Menjadi jembatan antara kebutuhan bisnis pengguna dan insinyur.

• Kontrol Proyek

Memberikan kerangka kerja terukur dalam manajemen kualitas & jadwal.

Peran Rekayasa Kebutuhan

• Studi Kelayakan

Memeriksa kontribusi sistem terhadap tujuan organisasi & anggaran.

• Elisitasi & Analisis

Mengumpulkan dan menyelaraskan kebutuhan stakeholder.

• Spesifikasi & Validasi

Menyusun dokumen SRS formal yang konsisten & dites.

Dua Paradigma Utama

• Pendekatan Terstruktur

Fokus pada pemisahan aliran proses (DFD) dan struktur data (ERD).

• Pendekatan Berorientasi Objek

Menggabungkan data & fungsi ke dalam objek terbungkus (UML).

• Evolusi

Berpindah dari pendekatan prosedural ke pendekatan pemodelan komponen dinamis.

Filosofi & Prinsip Utama Pendekatan Terstruktur

Prinsip Top-Down Decomposition

- Membagi-bagi program besar berdasarkan fungsi-fungsi atau prosedur-prosedur terurut.
- Fungsi/prosedur ditulis secara sekuensial dari atas ke bawah sesuai kebergantungan.
- Fungsi yang dipanggil harus dideklarasikan lebih dahulu di atasnya atau melalui file header.
- Program Utama (Main) mengontrol jalannya eksekusi dan memanggil sub-fungsi.

Pemisahan Pemodelan Proses & Data

- Pemodelan Proses

Menggambarkan transformasi aliran informasi menggunakan Data Flow Diagram (DFD).

- Pemodelan Data

Menggambarkan entitas data dan relasi menggunakan Entity Relationship Diagram (ERD).

- Keterbatasan

Perubahan pada struktur data dapat berdampak luas pada banyak fungsi yang mengakses data tersebut.

Pemodelan Proses: Data Flow Diagram (DFD)

Definisi & Konsep DFD

• Representasi grafik yang menggambarkan aliran informasi dan transformasi data dari masukan (input) hingga keluaran (output).

• Digunakan untuk memodelkan fungsi-fungsi sistem dan gerakan data antar proses.

• Prinsip Utama

Sistem beroperasi sebagai blok Input — Proses — Output.

• Memungkinkan pemodelan sistem pada beberapa tingkat abstraksi (leveling DFD).

Manfaat Pemodelan DFD

• Memberikan gambaran komprehensif mengenai batasan sistem (system boundary) dan entitas luar.

• Memudahkan analisis aliran data tanpa terikat oleh spesifikasi perangkat keras atau bahasa komputer tertentu.

• Menjadi landasan utama dalam penyusunan Spesifikasi Kebutuhan Fungsional SRS.

Empat Elemen Komponen Simbol DFD

1. Entitas Luar (External Entity)

• Sesuatu di luar sistem (orang, unit kerja, sistem lain) yang memberikan input data atau menerima output.

Penamaan: Kata Benda.

2. Aliran Data (Data Flow)

• Tempat mengalirnya data/informasi antar entitas, proses, dan data store. Digambarkan dengan garis berpanah.

Penamaan: Kata Benda.

3. Proses (Process)

• Menjelaskan apa yang dikerjakan sistem untuk mentransformasikan input menjadi output.

Penamaan: Kata Kerja + Kata Benda.

4. Penyimpanan Data (Data Store)

• Wadah penyimpanan data/informasi dalam sistem (tabel database/arsip).

Penamaan: Kata Benda (cth: transaksi_pembelian).

Aturan & Larangan Penting dalam Penggambaran DFD

Larangan Utama Aliran Data (Rules)

❌ Tidak Boleh Entitas ke Entitas: Aliran data dari entitas luar langsung ke entitas lain tanpa melalui proses.

❌ Tidak Boleh Entitas ke Data Store: Entitas luar tidak boleh berhubungan langsung dengan penyimpanan data tanpa proses.

❌ Tidak Boleh Store ke Store: Data store tidak boleh mengalirkan data langsung ke data store lain tanpa proses.

❌ Hindari Proses ke Proses Langsung: Aliran data antar proses sebaiknya melewati data store.

Aturan Penamaan & Kelengkapan Proses

✔ Setiap Proses Wajib Memiliki Input dan Output

Minimal memiliki 1 aliran data masuk dan 1 aliran data keluar (Mencegah Black Hole / Miracle).

✔ Nama Aliran Data Harus Spesifik

Hindari kata umum seperti 'data' atau 'informasi'.

✔ Identifikasi Proses

DFD Level 0 diberi nomor 1.0, 2.0; DFD Level 1 diberi nomor rincian 1.1, 1.2, dst.

Hirarki Leveling DFD & Konsep Keseimbangan (Balancing)

Tingkatan Level DFD

• Diagram Konteks

Diagram tingkat tertinggi yang menggambarkan lingkup keseluruhan sistem dan interaksi entitas luar.

• DFD Level 0

Mendetailkan proses-proses utama, entitas luar, data flow, dan data store utama.

• DFD Level 1

Mendetailkan sub-proses dari salah satu proses utama yang ada pada Level 0.

• DFD Level 2

Rincian lebih dalam dari sub-proses DFD Level 1 jika masih kompleks.

Prinsip Keseimbangan (Balancing)

• Definisi Balancing

Aliran data yang masuk/keluar pada suatu proses di level atas harus sama persis dengan level bawahnya.

• Kriteria Keseimbangan

Jumlah data flow sama, Nama data flow sama.

• Contoh

Jika pada DFD Level 0 proses 1.0 menerima input 'KRS' dan menghasilkan 'Bukti_krs', maka rincian DFD Level 1 (1.1, 1.2) total input dan outputnya harus persis sama.

Pemodelan Data: Entity Relationship Diagram (ERD)

Konsep Dasar ERD

- Penggambaran hubungan antar entitas data sebagai tahapan awal sebelum merancang basis data relasional.
- Dirancang agar mudah dipahami oleh pengguna non-teknis maupun tim pengembang.
- Tiga Komponen Dasar: Entitas (Persegi Panjang), Relationship/Relasi (Belah Ketupat), dan Atribut (Oval).

Tahapan Pembuatan ERD

- Tahap 1: Identifikasi entitas-entitas yang terlibat dalam sistem.
- Tahap 2: Lengkapi setiap entitas dengan atribut-atributnya dan tentukan Primary Key (PK).
- Tahap 3: Hubungkan antar entitas melalui hubungan relasi.
- Tahap 4: Tentukan derajat kardinalitas pada setiap relasi.

Klasifikasi Tipe Atribut dalam ERD

Key & Simple Attributes

- Key Attribute

Digunakan untuk menentukan suatu entitas secara unik (cth: NIM, NIK, kode_barang).

- Simple Attribute

Atribut bernilai tunggal yang tidak dapat dibagi lagi menjadi bagian yang lebih kecil (cth: nama, harga).

Multivalue & Composite

- Multivalue Attribute

Atribut yang memiliki sekelompok nilai pada setiap baris data (cth: gelar, hobi). Digambarkan dengan oval garis ganda.

- Composite Attribute

Atribut yang memiliki komponen lebih kecil dengan arti tertentu (cth: alamat -> jalan, kota, provinsi).

Derivative Attribute

- Derivative Attribute

Atribut yang dihasilkan dari hasil pengolahan/perhitungan atribut lain (cth: umur dihitung dari tgl_lahir dan tgl_sekarang).

- Notasi ERD

Digambarkan dengan oval bergaris putus-putus.

Strategi Penanganan Multivalue Attribute dalam Database

Masalah Multivalue Attribute

- Penggunaan Multivalue Attribute (seperti kolom 'gelar' berisi 'ST, M.Kom, PhD') sangat tidak disarankan dalam basis data relasional.
- Melanggar aturan normalisasi First Normal Form (1NF) yang mewajibkan setiap sel data bersifat atomik.
- Menyebabkan kesulitan dalam pencarian data, pembaruan, dan pengindeksan.

Solusi Pembentukan Entitas Asosiatif

- Langkah 1: Pecah atribut multivalue menjadi entitas master baru (cth: Entitas GELAR).
- Langkah 2: Buat relasi Many-to-Many (M:N) antara entitas utama (DOSEN) dan entitas master (GELAR).
- Langkah 3: Bentuk entitas/tabel penyambung (DOSEN_GELAR) yang memuat Foreign Key dari kedua entitas dan atribut posisi/urutan.

Kardinalitas & Derajat Relationship ERD

Derajat Relationship

• Unary Relationship

Menghubungkan satu entitas dengan dirinya sendiri (cth: PEGAWAI mengatur PEGAWAI/supervisor).

• Binary Relationship

Menghubungkan dua entitas berbeda (cth: PEGAWAI memiliki KENDARAAN).

• Ternary Relationship

Menghubungkan tiga entitas sekaligus (cth: PEGAWAI bekerja pada DEPARTEMEN di KOTA).

Jenis Kardinalitas (Mapping Constraints)

• One-to-One (1:1)

Setiap anggota entitas A berhubungan dengan maksimal 1 anggota entitas B (cth: Pegawai — Kendaraan Dinas).

• One-to-Many (1:M)

Satu anggota entitas A berhubungan dengan banyak anggota entitas B (cth: Fakultas — Jurusan).

• Many-to-Many (M:N)

Banyak anggota entitas A berhubungan dengan banyak anggota entitas B (cth: Mahasiswa — Mata Kuliah).

Aturan Transformasi ERD ke Skema Database Relasional

Aturan Relasi 1:1 & 1:M

• Relasi 1:1: Setiap entitas menjadi tabel. Foreign Key (FK) dapat ditambahkan pada salah satu tabel yang berelasi.

• Relasi 1:M: Tambahkan Foreign Key (FK) pada tabel anak (sisi Many/M). FK berasal dari Primary Key (PK) tabel induk (sisi 1).

Aturan Relasi M:N

• Relasi M:N: Wajib menghasilkan tabel baru (tabel persimpangan/junction table).

• Pembentukan FK: FK diambil dari kedua Primary Key entitas yang terhubung, membentuk Composite PK atau PK baru.

Contoh Konversi Karyawan

• Tabel JENIS_KARYAWAN: PK: Kode_jenis_karyawan.

• Tabel KARYAWAN: PK: Kode_karyawan. FK: Kode_jenis_karyawan (berasal dari relasi 1:M).

Implementasi Tingkat Kode: Pemrograman Terstruktur

Struktur Bahasa C++

- Menggunakan file header (#include <iostream>) untuk memanggil library fungsi.
- Deklarasi Fungsi: Fungsi seperti 'int hitung(int input)' dideklarasikan sebelum dipanggil.
- Fungsi Utama: 'int main()' bertindak sebagai pengontrol eksekusi sekuensial.
- Eksekusi Sub-Proses: Memanggil fungsi hitung(x) untuk memproses kalkulasi spesifik.

Pemrograman Terstruktur di PHP

- Memisahkan Kode: Menggunakan 'include_once "koneksi_db.php"' untuk pemisahan modul database.
- Pengapsulan Fungsi Prosedural: Membuat 'function query_sql($seleksi)' untuk memisahkan logika query dari kode tampilan.
- Perbandingan: Pemrograman tidak terstruktur mencampur kode koneksi, query, dan HTML dalam satu file (spaghetti code).

Filosofi Utama Pendekatan Berorientasi Objek (OO)

Prinsip Pemodelan Objek

• Menggabungkan data (atribut) dan fungsi (perilaku/method) ke dalam satu kesatuan terbungkus yang disebut Objek.

• Meniru cara manusia melihat komponen di dunia nyata.

• Objek Dunia Nyata

Memiliki status (state) dan perilaku (behavior).
Contoh: Objek Mahasiswa memiliki data NIM/Nama dan perilaku Registrasi/Presensi.

Keunggulan Utama Paradigma OO

• Modularitas Tinggi

Perubahan internal dalam satu objek tidak merusak objek lain.

• Reusabilitas (Pewarisan)

Kode class dapat diturunkan dan digunakan kembali tanpa membuat dari nol.

• Kemudahan Pemeliharaan

Pemetaan yang jelas antara masalah dunia nyata dan arsitektur perangkat lunak.

Empat Pilar Utama Pemrograman & Analisis OO

1. Enkapsulasi (Encapsulation)

Membungkus data dan metode dalam class serta menyembunyikan detail implementasi internal dari luar.

2. Pewarisan (Inheritance)

Mekanisme di mana class anak (subclass) mewarisi atribut dan metoda dari class induk (superclass).

3. Polimorfisme (Polymorphism)

Kemampuan objek bersikap dalam banyak bentuk, memungkinkan metoda yang sama memiliki perilaku berbeda pada class berbeda.

4. Abstraksi (Abstraction)

Menampilkan karakteristik penting dari suatu objek dan menyembunyikan kompleksitas detail eksekusi.

Sejarah & Standarisasi Unified Modeling Language (UML)

Evolusi Bahasa Visual UML

- 1970–1980: Pendekatan analisis dan perancangan berbasis OO mulai diperkenalkan.
- 1989–1994: Muncul metodologi terpisah: Booch Method (Grady Booch), OMT (James Rumbaugh), dan OOSE (Ivar Jacobson).
- Oktober 1994: Ketiga pakar ('Three Amigos') bergabung di Rational Software Co. untuk menyatukan metodologi.
- November 1997: UML secara resmi disetujui dan distandarisasi oleh OMG (Object Management Group).

Definisi & Scope UML

- UML adalah bahasa standar visual untuk menspesifikasikan, menggambarkan, membangun, dan mendokumentasikan artefak sistem perangkat lunak.
- Bukan Metodologi: UML menyampaikan 'bagaimana membuat model', namun tidak menentukan kapan dan apa model yang harus dibuat.
- Interoperabilitas: Dapat langsung dihubungkan ke bahasa OO (Java, C++, VB) maupun OODB.

Bangunan Dasar UML: Empat Kategori Things

A. Structural Things

Elemen statis fisik/konseptual model: Kelas, Antarmuka (Interface), Kolaborasi, Use Case, Kelas Aktif, Komponen, & Simpul (Node).

B. Behavioral Things

Elemen dinamis yang merepresentasikan perilaku dalam waktu dan ruang: Interaksi (Message) dan State Machine.

C. Grouping Things

Bagian pengorganisasi model UML untuk menyederhanakan kompleksitas (cth: Paket / Package).

D. Annotational Things

Bagian yang memperjelas dan memberi komentar/catatan pada elemen model UML.

Bangunan Dasar UML: Empat Jenis Relationships

Dependency & Association

• Dependency (Kebergantungan)

Hubungan di mana perubahan pada elemen independen mempengaruhi elemen lain. Notasi: Panah putus-putus.

• Asosiasi (Association)

Menghubungkan antar objek. Bentuk khususnya adalah Agregasi (hubungan bagian/part-of). Notasi: Garis tegas.

Generalization & Realization

• Generalisasi (Generalization)

Hubungan hirarkis di mana objek anak berbagi perilaku/struktur dari objek induk (Inheritance). Notasi: Panah kepala segitiga kosong.

• Realisasi (Realization)

Operasi yang benar-benar dilakukan oleh suatu objek/komponen dalam memenuhi spesifikasi interface. Notasi: Panah putus-putus kepala segitiga kosong.

Overview Sembilan Diagram Utama Standar UML

Diagram Pandangan Statis (Structural)

1. Use Case Diagram: Kebutuhan fungsional sistem dari sudut pandang pemakai.
2. Class Diagram: Struktur statis kelas, atribut, metode, dan relasi.
3. Object Diagram: Instansiasi kelas pada momen tertentu saat eksekusi.
4. Component Diagram: Organisasi dan kebergantungan komponen fisik software.
5. Deployment Diagram: Konfigurasi fisik node hardware & runtime.

Diagram Pandangan Dinamis (Behavioral)

6. Sequence Diagram: Interaksi antar objek dalam urutan waktu sekuensial.
7. Collaboration Diagram: Organisasi struktural objek yang mengirim/menerima pesan.
8. Statechart Diagram: Perubahan state/status objek akibat kejadian (events).
9. Activity Diagram: Alur kerja (workflow) proses bisnis dan aktivitas.

Use Case Diagram: Pemodelan Kebutuhan Fungsional

Elemen Utama Use Case Diagram

Use Case: Menggambarkan pola perilaku/fungsionalitas sistem yang menghasilkan hasil terukur bagi Actor. Notasi: Elips horizontal. Penamaan: Kata Kerja.

Actor: Orang, sistem luar, atau entitas yang menyediakan/menerima informasi. Notasi: Stickman. Penamaan: Kata Benda.

System Boundary Box: Kotak batas yang mendefinisikan jangkauan/scope sistem.

Prinsip Pemodelan Use Case

Fokus pada 'APA' yang dilakukan sistem, bukan 'BAGAIMANA' sistem mengerjakannya.

Tidak Boleh Ada Komunikasi Langsung Antar Actor.

Actor Utama diletakkan pada pojok kiri atas diagram.

Menggambarkan urutan transaksi yang berhubungan untuk memenuhi tujuan pengguna.

Relasi Use Case: <<include>>, <<extend>>, & Generalisasi

<<include>> Relationship

- Ketergantungan Wajib (Required).
- Pemanggilan fungsi oleh use case lain (cth: Buka Rekening <<include>> Catat Data Pribadi).
- Panah terbuka terarah dari base use case ke sub use case yang di-include.

<<extend>> Relationship

- Perluasan Opsional (Conditional) jika syarat terpenuhi.

Contoh: Buka Rekening <<extend>> Buka Deposito.

- Panah terbuka terarah dari extension use case kembali ke base/parent use case.

Generalisasi Use Case/Actor

- Digunakan ketika ada perlakuan khusus/spesialisasi.
- Panah ber-ujung segitiga tertutup mengarah ke elemen yang lebih umum.

Class Diagram: Pemodelan Struktur Statis Sistem

Tiga Area Utama Komponen Class

1. Nama Class: Nama unik yang mendefinisikan kelompok objek (cth: Mahasiswa, Dosen).
2. Atribut / Properti: Mendeskripsikan keadaan/state data yang dimiliki oleh class.
3. Metoda / Operasi: Layanan atau fungsi yang ditawarkan untuk memanipulasi keadaan data.

Tingkat Aksesibilitas (Visibility)

Private (-): Atribut/metoda tidak dapat dipanggil dari luar class yang bersangkutan.

Protected (#): Hanya dapat dipanggil oleh class bersangkutan dan subclass turunannya.

Public (+): Dapat dipanggil dan diakses oleh class mana saja dalam sistem.

Relasi & Multiplisitas pada Class Diagram

Jenis Relasi Antar Class

- Asosiasi (Association): Hubungan statis antar class di mana satu class memiliki atribut berupa class lain.
- Agregasi (Aggregation): Hubungan kepemilikan 'terdiri atas' (part-of) di mana komponen dapat berdiri sendiri.
- Pewarisan (Inheritance): Hubungan hirarkis di mana subclass mewarisi atribut dan metoda dari superclass.

Notasi Multiplisitas (Multiplicity)

- 1: Tepat satu instansi.
- 0..1: Nol atau satu instansi (opsional).
- * atau 0..*: Nol atau banyak instansi.
- 1..*: Satu atau banyak instansi.
- n..m: Rentang spesifik tertentu (cth: 2..4).

Activity Diagram: Pemodelan Alur Kerja Bisnis

Konsep & Simbol Utama

- Start / End Point: Titik awal (lingkaran hitam) dan akhir (lingkaran hitam ganda) proses.
- Activities: Langkah aktivitas atau eksekusi proses.
- Decision Point: Percabangan kondisi logika (belah ketupat).
- Fork & Join: Pembagian alur paralel (Fork) dan penggabungan kembali alur paralel (Join).

Penggunaan Swimlanes

- Definisi Swimlane: Kolom pengelompokkan aktivitas berdasarkan Actor atau unit organisasi yang bertanggung jawab.
- Manfaat: Memperlihatkan secara jelas pembagian wewenang dan alur koordinasi antar aktor (cth: Customer, ATM Machine, Bank).
- Mirip dengan flowchart atau DFD namun dengan fokus pada eksekusi proses bisnis berurutan.

Sequence Diagram: Interaksi Objek Berbasis Waktu

Elemen Penting Sequence Diagram

- Object Lifeline: Garis putus-putus vertikal yang menandakan keberadaan objek selama urutan interaksi.
- Activation Box: Persegi panjang sempit pada lifeline yang menunjukkan durasi waktu objek aktif mengeksekusi proses.
- Synchronous Message: Pesan langsung dengan panah solid di mana pengirim menunggu respon.
- Return Message: Pesan balasan dengan garis putus-putus.

Karakteristik Pemodelan Dinamis

- Dibuat berdasarkan skenario spesifik dari satu Use Case.
- Menekankan pada urutan kronologis waktu pengiriman pesan antar objek.
- Contoh Skenario: Pengiriman SMS Akademik -> Validasi Format -> Validasi User -> Balasan Pesan.

Diagram Interaksi & Perilaku: Collaboration & Statechart

Collaboration Diagram (Interaction)

- Diagram interaksi dinamis yang menekankan pada organisasi struktural objek-objek.
- Memperlihatkan keterhubungan antar objek beserta nomor urut pesan yang dikirim/diterima.
- Perbedaan vs Sequence: Sequence menekankan aspek waktu; Collaboration menekankan aspek arsitektur hubungan objek.

Statechart Diagram (State Machine)

- Menggambarkan seluruh status (state) yang dimiliki oleh suatu objek dari suatu class.
- Memperlihatkan kejadian (events) dan kondisi transisi yang menyebabkan perubahan status objek.
- Sangat berguna untuk memodelkan objek yang memiliki lifecycle kompleks (cth: Status Transaksi: Draft -> Validasi -> Dibayar -> Selesai).

Diagram Arsitektur Fisik: Component & Deployment

Component Diagram (Software Structure)

- Diagram statis yang memperlihatkan organisasi dan kebergantungan komponen-komponen perangkat lunak.
- Komponen dapat berupa modul kode, library, database, atau executable file.
- Menghubungkan arsitektur software melalui antarmuka (interface) dan kebergantungan (dependency).

Deployment Diagram (Hardware & Runtime)

- Diagram statis yang memperlihatkan konfigurasi fisik hardware saat aplikasi dijalankan (runtime).
- Memuat Simpul (Node) komputasi seperti Web Server, Application Server, Database Server, dan Client PC.
- Menggambarkan pemetaan komponen software ke dalam node fisik perangkat keras.

Matriks Komparasi Utama: Terstruktur vs. Berorientasi Objek

Dimensi Perbandingan	Pendekatan Terstruktur	Pendekatan Berorientasi Objek
Fokus Utama	Alur proses sekuensial & fungsi terurut	Objek gabungan data & perilaku
Pemisahan Data & Proses	Dua pemodelan terpisah (DFD & ERD)	Menyatu dalam Class & Objek (UML)
Alat Pemodelan Standard	Data Flow Diagram & ERD	Unified Modeling Language (UML)
Fleksibilitas Perubahan	Kaku (perubahan data berdampak luas)	Tinggi (dampak terisolasi dalam objek)
Reusabilitas Kode	Rendah (fungsi prosedural terikat)	Tinggi (Pewarisan & Polymorphism)
Orientasi Sistem	Sistem berbasis fungsi prosedural linier	Sistem berbasis komponen & event-driven

Kriteria Pemilihan Metodologi Pengembangan

1. Stabilitas Kebutuhan (Requirements Stability)

Spesifikasi jelas & stabil cocok untuk metode Terstruktur/Waterfall. Kebutuhan dinamis & sering berubah cocok untuk OO/Agile.

2. Skala & Kompleksitas Sistem

Proyek sederhana dengan lingkup terbatas cocok untuk RAD/Waterfall. Sistem enterprise kompleks cocok untuk RUP/OO/Agile.

3. Keterlibatan Pengguna (User Involvement)

Keterlibatan di awal/akhir proyek vs keterlibatan intensif secara berulang di setiap siklus iterasi.

4. Batasan Waktu & Manajemen Risiko

Target time-to-market sangat cepat (RAD/Agile) vs analisis risiko ketat proyek kritis (Spiral Model).

Pembahasan Contoh Model SDLC Klasik & Iteratif

Waterfall Model

Model linier sekuensial (Analisis -> Desain -> Koding -> Pengujian). Cocok jika kebutuhan sangat stabil dan terdokumentasi rapi.

V-Model

Perluasan Waterfall yang memasangkan setiap tahap pembangunan dengan tahap verifikasi/validasi pengujian (Requirements <-> Acceptance Test).

RAD (Rapid App Dev)

Fokus pada pembuatan prototipe cepat (60-90 hari) dengan Joint Application Development (JAD) intensif bersama pengguna.

Spiral & RUP Model

Spiral: Berbasis analisis risiko iteratif. RUP: Kerangka berorientasi objek berbasis Use Case melalui 4 fase (Inception, Elaboration, Construction, Transition).

Metodologi Modern: Agile Development (Scrum & Kanban)

Kerangka Kerja Scrum

• Pendekatan Adaptif

Mengembangkan sistem dalam siklus pendek (Sprint 1-4 minggu) yang menghasilkan produk berfungsi.

• Peran Utama

Product Owner (pemilik kebutuhan), Scrum Master (fasilitator), dan Development Team.

• Seremonial Scrum

Sprint Planning, Daily Standup (15 menit), Sprint Review, dan Sprint Retrospective.

• Kelebihan

Sangat responsif terhadap perubahan kebutuhan di tengah proses rekayasa.

Metode Kanban & Continuous Delivery

• Visualisasi Alur Kerja

Menggunakan papan Kanban (To Do, In Progress, Testing, Done) untuk memantau status tugas.

• Pembatasan WIP

Membatasi Work-in-Progress (WIP) untuk mengidentifikasi bottleneck dan meningkatkan kecepatan penyerahan.

• Continuous Delivery

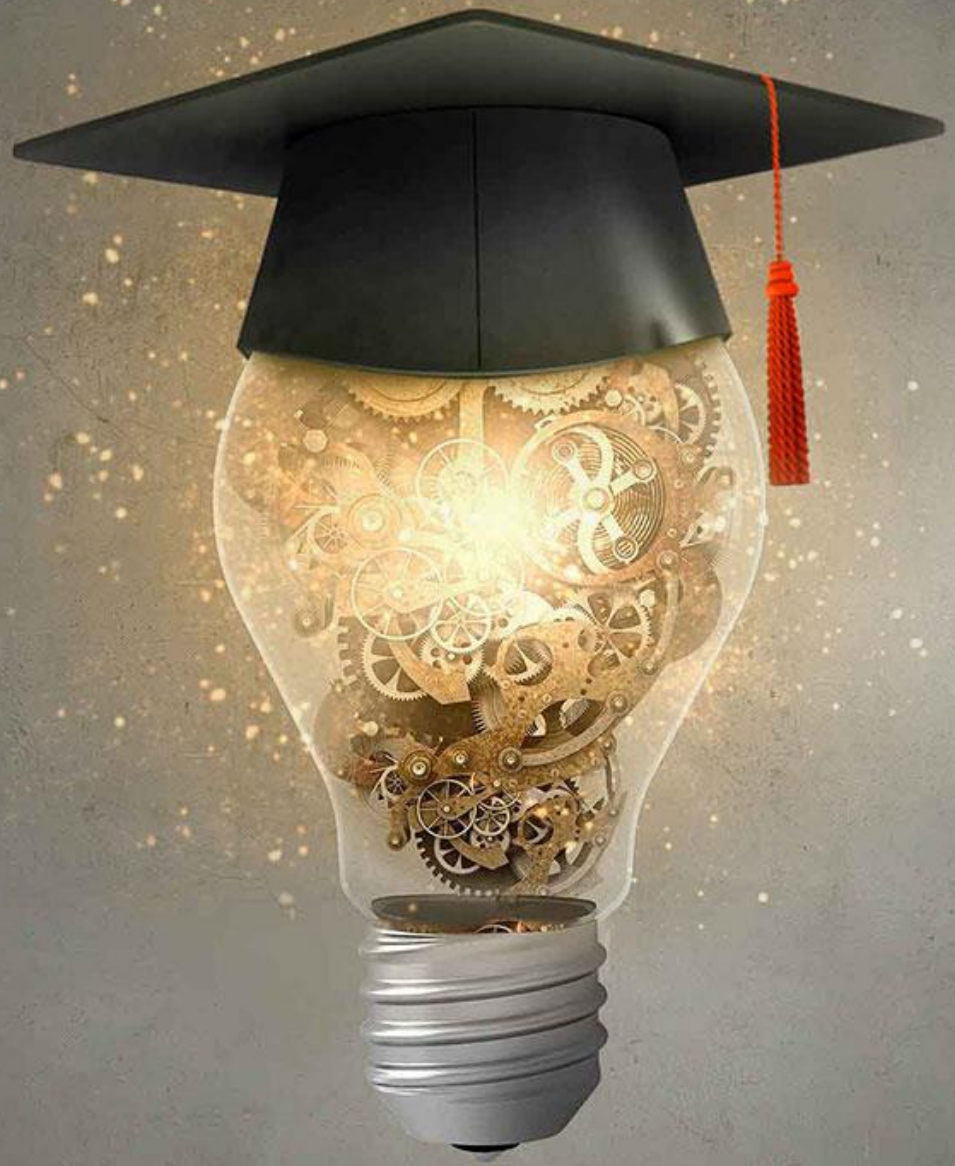
Perubahan kebutuhan dapat langsung diprioritaskan dan diimplementasikan tanpa menunggu siklus rilis besar.

MATRIKS KEPUTUSAN & TUGAS STUDI KASUS

Rangkuman Panduan SDLC

Panduan Ringkas Pemilihan SDLC

- Waterfall: Kebutuhan baku, risiko rendah, teknologi sangat dikuasai.
- RAD: Waktu mendesak (<90 hari), sistem berbasis komponen.
- Spiral: Sistem skala besar dengan tingkat risiko tinggi.
- RUP / OO: Sistem berorientasi objek dengan pemodelan UML lengkap.
- Agile / Scrum: Kebutuhan belum pasti, berorientasi inovasi cepat.



THANK YOU